Universidad Politécnica de Cartagena
Escuela Técnica Superior de Ingeniería de Telecomunicación

Pico Miner

Acelerador de Minería Bitcoin basado en FPGA

FPGA-Based Bitcoin Mining Accelerator

SHA-256 Doble con Optimización de Midstate
Double-SHA-256 with Midstate Optimization

Plataforma / Platform:	Xilinx Zynq-7020 (ZedBoard)
Herramienta / Tool:	Vivado HLS 2019.1
Bloque demo / Demo block:	Bitcoin Block #939260
Fecha / Date:	Marzo / March 2026

Memoria del Proyecto / Project Memory

Contents

1 Introducción / Introduction **2**

2 Fundamentos: Minería Bitcoin / Background: Bitcoin Mining **2**
- 2.1 La Blockchain / The Blockchain . . . 2
- 2.2 Cabecera de Bloque de 80 Bytes / The 80-Byte Block Header . . . 3
- 2.3 Prueba de Trabajo / Proof of Work . . . 3
- 2.4 Ventajas de las FPGAs / Why FPGAs? . . . 3

3 Algoritmo SHA-256 / SHA-256 Algorithm **4**
- 3.1 Visión General / Overview . . . 4
- 3.2 Relleno del Mensaje / Message Padding . . . 4
- 3.3 Función de Compresión / Compression Function . . . 4
- 3.4 Doble SHA-256 para Bitcoin / Double SHA-256 for Bitcoin . . . 5

4 Optimización de Midstate / Midstate Optimization **6**
- 4.1 Idea Clave / Key Insight . . . 6
- 4.2 Partición ARM/FPGA / ARM/FPGA Partitioning . . . 6

5 Arquitectura del Sistema / System Architecture **7**
- 5.1 Interfaz del IP HLS / HLS IP Interface . . . 7
- 5.2 Interfaz de Registros AXI-Lite / AXI-Lite Register Interface . . . 7
- 5.3 Flujo de Operación de Minería / Mining Operation Flow . . . 7
- 5.4 Estrategia de Minería por Bloques / Chunked Mining Strategy . . . 8
- 5.5 Control de LEDs / LED Control . . . 9

6 Estrategia de Optimización HLS / HLS Optimization Strategy **9**
- 6.1 Función de Compresión SHA-256 / SHA-256 Compress Function . . . 9
- 6.2 Bucle de Minería / Mining Loop . . . 10
- 6.3 Configuraciones de Solución / Solution Configurations . . . 10

7 Bloque 939260 – Demostración / Block 939260 – Demo **11**
- 7.1 Orden de Bytes del Nonce / Nonce Byte Order . . . 11

8 Verificación y Pruebas / Verification and Testing **12**
- 8.1 Testbench HLS (5 Pruebas) / HLS Testbench (5 Tests) . . . 12
- 8.2 Simulación C (csim) / C Simulation (csim) . . . 12
- 8.3 Co-Simulación C/RTL (cosim) / C/RTL Co-Simulation (cosim) . . . 12
- 8.4 Verificación en ARM / ARM Verification . . . 13

9 Resultados / Results **13**
- 9.1 Rendimiento Estimado / Estimated Performance . . . 13
- 9.2 Tiempo de Minería del Bloque 939260 / Block 939260 Mining Time . . . 13
- 9.3 Comprobación de Dificultad / Difficulty Check . . . 14

10 Guía de Reproducción / Reproduction Guide **14**
10.1 Paso 1: Vivado HLS / Step 1: Vivado HLS 14
10.2 Paso 2: Vivado – Diseño de Bloques / Step 2: Vivado – Block Design 15
10.3 Paso 3: Xilinx SDK / Step 3: Xilinx SDK 15

11 Conclusión / Conclusion **16**

12 Archivos del Proyecto / Project Files **17**

1 Introducción / Introduction

ES: La minería de criptomonedas es el proceso mediante el cual se verifican las transacciones y se añaden al registro distribuido conocido como *blockchain*. El reto computacional central se denomina **Prueba de Trabajo** (*Proof of Work*, PoW): los mineros deben encontrar un valor llamado *nonce* tal que el hash del bloque de datos cumpla una condición de dificultad — típicamente, el hash resultante debe ser numéricamente menor que un valor *objetivo* (*target*).

EN: *Cryptocurrency mining is the process by which transactions are verified and added to the distributed ledger known as the blockchain. The core computational challenge is called* ***Proof of Work*** *(PoW): miners must find a value called a nonce such that the hash of the block data satisfies a difficulty condition — typically, the resulting hash must be numerically less than a target value.*

ES: En Bitcoin, esto implica calcular la función hash SHA-256 dos veces (*double SHA-256*) sobre una cabecera de bloque de 80 bytes, iterando sobre un espacio de nonces de 32 bits. El objetivo de dificultad se ajusta para que, en promedio, se encuentre un nonce válido cada 10 minutos en toda la red.

EN: *In Bitcoin, this involves computing the SHA-256 hash function twice (double SHA-256) over an 80-byte block header while iterating over a 32-bit nonce space. The difficulty target is adjusted so that, on average, a valid nonce is found every 10 minutes across the entire network.*

ES: **Pico Miner** implementa la minería Bitcoin double-SHA-256 con optimización de midstate, acelerada en una FPGA Zynq-7020 mediante Vivado HLS 2019.1. La demostración mina el Bloque 939260 de Bitcoin (4 de marzo de 2026, dificultad 144.398.401.518.101) desde nonce=0 en aproximadamente 3 minutos, con retroalimentación en tiempo real por terminal UART y LEDs del ZedBoard.

EN: *Pico Miner implements Bitcoin double-SHA-256 mining with midstate optimization, accelerated on a Zynq-7020 FPGA using Vivado HLS 2019.1. The demo mines Bitcoin Block 939260 (March 4, 2026, difficulty 144,398,401,518,101) from nonce=0 in approximately 3 minutes, with real-time feedback via UART terminal and ZedBoard LEDs.*

2 Fundamentos: Minería Bitcoin / Background: Bitcoin Mining

2.1 La Blockchain / The Blockchain

ES: Una blockchain es un registro distribuido y de solo escritura, donde cada bloque contiene:

- Una referencia (hash) al bloque anterior
- Un conjunto de transacciones (representadas por la raíz de Merkle)
- Una marca de tiempo (*timestamp*) y bits de dificultad

- Un valor *nonce*

EN: *A blockchain is a distributed, append-only ledger where each block contains: a reference (hash) to the previous block, a set of transactions (represented by the Merkle root), a timestamp and difficulty bits, and a nonce value.*

2.2 Cabecera de Bloque de 80 Bytes / The 80-Byte Block Header

ES: Cada cabecera de bloque de Bitcoin consta de exactamente 80 bytes (640 bits):
EN: *Every Bitcoin block header consists of exactly 80 bytes (640 bits):*

Table 1: Estructura de la cabecera de bloque Bitcoin / Bitcoin Block Header Structure

2palebluewhite

Campo / Field	**Bytes**	**Descripción**	**Description**
Version	4	Número de versión	Block version number
Previous hash	32	Hash del bloque anterior	Hash of previous block
Merkle root	32	Hash de las transacciones	Hash of all transactions
Timestamp	4	Marca de tiempo Unix	Unix timestamp
Bits	4	Representación compacta	Compact target
Nonce	4	Valor iterado por mineros	Value iterated by miners

2.3 Prueba de Trabajo / Proof of Work

ES: El mecanismo de PoW requiere que:

$$\text{SHA-256}\big(\text{SHA-256}(\text{cabecera_bloque})\big) < \text{objetivo} \tag{1}$$

Dado que SHA-256 se comporta como un oráculo pseudo-aleatorio, la única forma de encontrar un nonce válido es mediante **iteración por fuerza bruta**: probando valores de nonce uno por uno hasta cumplir la condición.
EN: *The PoW mechanism requires that SHA-256(SHA-256(block_header)) < target. Since SHA-256 behaves as a pseudo-random oracle, the only way to find a valid nonce is by brute-force iteration — trying nonce values one by one until the condition is met.*

2.4 Ventajas de las FPGAs / Why FPGAs?

ES: Las FPGAs ofrecen varias ventajas para la minería PoW:

- **Paralelismo**: Múltiples cálculos de hash simultáneos en hardware
- **Pipeline**: Las rondas de compresión se pueden segmentar profundamente
- **Eficiencia energética**: Hardware dedicado es más eficiente que CPUs o GPUs

- **Baja latencia**: Implementación directa sin overhead de búsqueda/decodificación de instrucciones

EN: *FPGAs offer parallelism (multiple simultaneous hash computations), deep pipelining of compression rounds, energy efficiency compared to CPUs/GPUs, and low latency through direct hardware implementation without instruction fetch/decode overhead.*

3 Algoritmo SHA-256 / SHA-256 Algorithm

3.1 Visión General / Overview

ES: SHA-256 (Secure Hash Algorithm, 256 bits) está definido en FIPS 180-4. Procesa la entrada en bloques de 512 bits (64 bytes), manteniendo un estado de ocho palabras de 32 bits (256 bits en total) que se actualiza mediante una función de compresión de 64 rondas.

EN: *SHA-256 (Secure Hash Algorithm, 256-bit) is defined in FIPS 180-4. It processes input in 512-bit (64-byte) blocks, maintaining an eight-word (256-bit) state that is updated by a compression function consisting of 64 rounds.*

3.2 Relleno del Mensaje / Message Padding

ES: El mensaje de entrada se rellena hasta ser múltiplo de 512 bits:

1. Añadir un bit `1`
2. Añadir bits `0` hasta que la longitud sea $\equiv 448 \pmod{512}$
3. Añadir la longitud original del mensaje como un entero de 64 bits en big-endian

Para la cabecera Bitcoin de 80 bytes, el mensaje rellenado tiene 1024 bits (dos bloques de 512 bits).

EN: *The input message is padded to a multiple of 512 bits by appending a `1` bit, then `0` bits until the length is* $\equiv 448 \pmod{512}$*, and finally the original message length as a 64-bit big-endian integer. For the 80-byte Bitcoin header, the padded message is 1024 bits (two 512-bit blocks).*

3.3 Función de Compresión / Compression Function

ES: Cada bloque de 512 bits se procesa según el Algoritmo 1. Las funciones SHA-256 utilizadas son:

EN: *Each 512-bit block is processed according to Algorithm 1. The SHA-256 functions used are:*

Algorithm 1 SHA-256 Compress / Compresión SHA-256

Input / Entrada: Estado $H[0..7]$, Bloque $M[0..15]$
Expansión del calendario de mensajes / Message schedule:
for $i = 0$ **to** 15 **do**
  $W[i] \leftarrow M[i]$
end for
for $i = 16$ **to** 63 **do**
  $W[i] \leftarrow \sigma_1(W[i-2]) + W[i-7] + \sigma_0(W[i-15]) + W[i-16]$
end for
Inicializar variables / Initialise working variables:
$a, b, c, d, e, f, g, h \leftarrow H[0], H[1], \ldots, H[7]$
64 rondas de compresión / 64 compression rounds:
for $i = 0$ **to** 63 **do**
  $T_1 \leftarrow h + \Sigma_1(e) + \mathrm{Ch}(e, f, g) + K[i] + W[i]$
  $T_2 \leftarrow \Sigma_0(a) + \mathrm{Maj}(a, b, c)$
  $h \leftarrow g;\ g \leftarrow f;\ f \leftarrow e;\ e \leftarrow d + T_1$
  $d \leftarrow c;\ c \leftarrow b;\ b \leftarrow a;\ a \leftarrow T_1 + T_2$
end for
Actualizar estado / Update state:
$H[i] \leftarrow H[i] + \{a, b, c, d, e, f, g, h\}$ for $i = 0..7$

$$\mathrm{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g) \tag{2}$$
$$\mathrm{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c) \tag{3}$$
$$\Sigma_0(a) = \mathrm{ROTR}^2(a) \oplus \mathrm{ROTR}^{13}(a) \oplus \mathrm{ROTR}^{22}(a) \tag{4}$$
$$\Sigma_1(e) = \mathrm{ROTR}^6(e) \oplus \mathrm{ROTR}^{11}(e) \oplus \mathrm{ROTR}^{25}(e) \tag{5}$$
$$\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x) \tag{6}$$
$$\sigma_1(x) = \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x) \tag{7}$$

3.4 Doble SHA-256 para Bitcoin / Double SHA-256 for Bitcoin

ES: La minería Bitcoin calcula:

$$\text{hash} = \text{SHA-256}\big(\text{SHA-256}(\text{cabecera}_{80\text{B}})\big)$$

El primer SHA-256 procesa la cabecera de 80 bytes (rellenada a 128 bytes = 2 bloques). El segundo SHA-256 procesa el resultado de 32 bytes (rellenado a 64 bytes = 1 bloque). Total sin optimización: **3 compresiones** por nonce.

EN: *Bitcoin mining computes hash = SHA-256(SHA-256(header$_{80B}$)). The first SHA-256 processes the 80-byte header (padded to 128 bytes = 2 blocks). The second SHA-256 processes the resulting 32-byte digest (padded to 64 bytes = 1 block). Total without optimization: 3 compressions per nonce.*

4 Optimización de Midstate / Midstate Optimization

4.1 Idea Clave / Key Insight

ES: La cabecera de 80 bytes abarca dos bloques de entrada SHA-256:

- **Chunk 1** (bytes 0–63): versión + hash previo + raíz de Merkle bytes 0–27
- **Chunk 2** (bytes 64–79 + relleno): raíz de Merkle bytes 28–31 + timestamp + bits + **nonce**

Dado que el nonce está en el chunk 2, el estado SHA-256 tras procesar el chunk 1 es **constante** para todos los intentos de nonce. Este estado intermedio se denomina *midstate*.

EN: *The 80-byte header spans two SHA-256 input blocks. Chunk 1 (bytes 0–63) contains version + previous hash + merkle root bytes 0–27. Chunk 2 (bytes 64–79 + padding) contains merkle root bytes 28–31 + timestamp + bits + nonce. Since the nonce is in chunk 2, the SHA-256 state after processing chunk 1 is constant across all nonce attempts. This intermediate state is called the midstate.*

Table 2: División de la cabecera para midstate / Header partitioning for midstate

2palebluewhite

Chunk	Bytes	Contenido	Procesado por / Processed b
Chunk 1	0–63	version + prev_hash + merkle[0..27]	ARM (una vez / once)
Chunk 2	64–79 + padding	merkle[28..31] + ts + bits + **nonce**	FPGA (por nonce / per nonce)

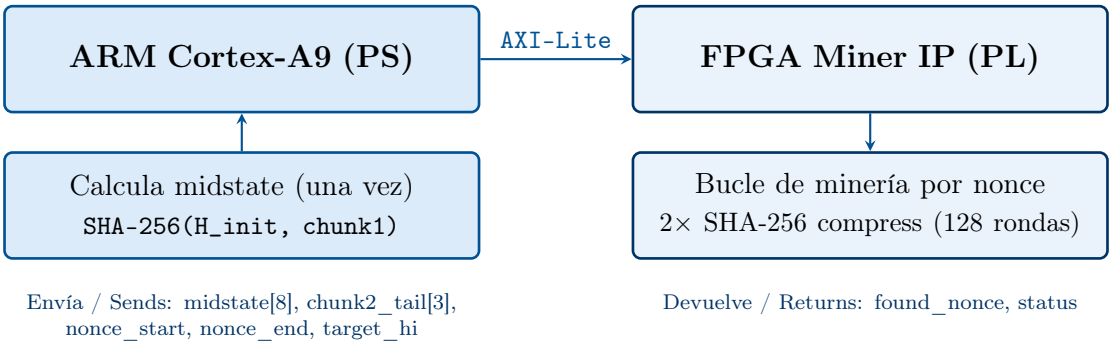
4.2 Partición ARM/FPGA / ARM/FPGA Partitioning

ES: El ARM calcula el midstate **una sola vez**, y luego la FPGA realiza únicamente:

1. 64 rondas de compresión para el chunk 2 (partiendo del midstate)
2. 64 rondas de compresión para el segundo SHA-256

Esto reduce el trabajo por nonce de 3 compresiones a **2 compresiones** (128 rondas en total), una reducción del 33%.

EN: *The ARM computes the midstate once, then the FPGA performs only 64 compression rounds for chunk 2 (starting from the midstate) and 64 compression rounds for the second SHA-256. This reduces the per-nonce work from 3 compressions to 2 compressions (128 rounds total), a 33% reduction.*



5 Arquitectura del Sistema / System Architecture

5.1 Interfaz del IP HLS / HLS IP Interface

ES: La firma de la función HLS del IP es:

EN: *The HLS IP function signature is:*

```
void pico_miner(
    unsigned int midstate[8],       // Estado SHA-256 precalculado (
        midstate)
    unsigned int chunk2_tail[3],    // merkle_tail, timestamp, bits
    unsigned int nonce_start,       // Inicio del rango de nonces
    unsigned int nonce_end,         // Fin del rango (exclusivo)
    unsigned int target_hi,         // MSB del objetivo de dificultad
    unsigned int *found_nonce,      // Salida: nonce encontrado
    unsigned int *status            // Salida: 1 = encontrado, 0 = no
        encontrado
);
```

ES: Todos los puertos se mapean a AXI-Lite (`s_axilite`) agrupados en un bus llamado `myaxi`. Vivado HLS 2019.1 mapea los argumentos de tipo array como registros escalares individuales:

- `midstate[8]` → `Set_midstate_0` ... `Set_midstate_7`
- `chunk2_tail[3]` → `Set_chunk2_tail_0` ... `Set_chunk2_tail_2`

EN: *All ports are mapped to AXI-Lite (s_axilite) bundled into a single bus called myaxi. Vivado HLS 2019.1 maps array arguments as individual scalar registers.*

5.2 Interfaz de Registros AXI-Lite / AXI-Lite Register Interface

5.3 Flujo de Operación de Minería / Mining Operation Flow

ES:

1. El ARM analiza la cabecera de 80 bytes y la convierte a big-endian

Table 3: Registros AXI-Lite / AXI-Lite Registers

2palebluewhite

Dirección / Direction	**Register**	**SDK Driver Function**
Input	midstate[0..7]	`XPico_miner_Set_midstate_0 .. _7`
Input	chunk2_tail[0..2]	`XPico_miner_Set_chunk2_tail_0 .. _2`
Input	nonce_start	`XPico_miner_Set_nonce_start`
Input	nonce_end	`XPico_miner_Set_nonce_end`
Input	target_hi	`XPico_miner_Set_target_hi`
Output	found_nonce	`XPico_miner_Get_found_nonce`
Output	status	`XPico_miner_Get_status`

2. El ARM calcula el midstate: `sha256_compress(H_init, chunk1)`
3. El ARM escribe midstate[8], chunk2_tail[3], rango de nonces y target en la FPGA vía AXI-Lite
4. El ARM envía `ap_start`
5. La FPGA itera nonces:
   (a) Construye el bloque de mensaje del chunk 2 (tail + nonce + relleno SHA-256)
   (b) Comprime chunk 2 con el midstate → primer hash
   (c) Hashea el primer hash con valores iniciales SHA-256 → hash final
   (d) Si `final_hash[7] <= target_hi`: almacena nonce, establece status
6. El ARM sondea `ap_done`, luego lee `found_nonce` y `status`

EN: *1. ARM parses the 80-byte header and byte-swaps to big-endian. 2. ARM computes midstate. 3. ARM writes parameters to FPGA via AXI-Lite. 4. ARM issues ap_start. 5. FPGA iterates nonces, performing double-SHA-256 and checking difficulty. 6. ARM polls ap_done, then reads found_nonce and status.*

5.4 Estrategia de Minería por Bloques / Chunked Mining Strategy

ES: Para la demostración, el driver ARM divide la búsqueda completa de nonces en fragmentos (*chunks*) de 1.000.000 de nonces por invocación del hardware. Entre fragmentos, el ARM:

- Imprime el progreso (nonces buscados, porcentaje, tiempo transcurrido, tasa de hash) por UART
- Actualiza la barra de progreso de LEDs (8 LEDs del ZedBoard se encienden proporcionalmente)
- Comprueba si la FPGA ha encontrado un nonce válido

Esto proporciona retroalimentación en tiempo real durante los ∼3 minutos de minería.

EN: *For the demo, the ARM driver breaks the full nonce search into chunks of 1,000,000 nonces per FPGA invocation. Between chunks, the ARM prints progress to UART, updates the LED progress bar (8 LEDs proportional to search progress), and checks whether the FPGA found a valid nonce. This provides real-time feedback during the approximately 3-minute mining run.*

5.5 Control de LEDs / LED Control

ES: Los 8 LEDs del ZedBoard se controlan mediante un bloque AXI GPIO:

- **Durante la minería**: Los LEDs se encienden proporcionalmente al progreso de búsqueda (0–8 LEDs)
- **Al completar**: Los 8 LEDs se encienden cuando el bloque se ha minado con éxito
- **Animación de inicio**: Doble parpadeo de todos los LEDs al comenzar la minería

El driver utiliza `XGpio_DiscreteWrite()` con `XPAR_AXI_GPIO_0_DEVICE_ID`.

EN: *The 8 ZedBoard LEDs are controlled via an AXI GPIO block. During mining, LEDs light up proportionally to search progress (0–8 LEDs). All 8 LEDs turn on when the block is successfully mined. A double-blink animation signals mining start.*

6 Estrategia de Optimización HLS / HLS Optimization Strategy

6.1 Función de Compresión SHA-256 / SHA-256 Compress Function

ES: La función `sha256_compress()` es el núcleo computacional crítico. Las optimizaciones clave aplicadas son:

EN: *The sha256_compress() function is the critical inner kernel. Key optimizations applied are:*

```
static void sha256_compress(const unsigned int state_in[8],
                            const unsigned int W_in[16],
                            unsigned int state_out[8])
{
#pragma HLS INLINE off
    unsigned int W[64];
#pragma HLS ARRAY_PARTITION variable=W complete dim=1

    // Carga + expansion de W (completamente desenrollado)
    // 64 rondas de compresion (pipeline II=1)
    compress:
    for (i = 0; i < 64; i++) {
#pragma HLS PIPELINE II=1
        // ... calculo de ronda / round computation
    }
}
```

ES:

- `ARRAY_PARTITION complete` en `W[64]`: las 64 palabras accesibles en paralelo (sin contención de puertos de memoria)
- `UNROLL` en carga y expansión de W: calculado combinacionalmente
- `PIPELINE II=1` en el bucle de 64 rondas: una ronda por ciclo de reloj
- `INLINE off`: mantiene la función compress como módulo reutilizable (se llama dos veces por nonce)

EN: *ARRAY_PARTITION complete on W[64] enables parallel access. UNROLL on W load/expansion computes combinationally. PIPELINE II=1 on the 64-round loop produces one round per clock cycle. INLINE off keeps the compress function as a reusable module (called twice per nonce).*

6.2 Bucle de Minería / Mining Loop

ES: El bucle de minería de nivel superior no tiene pragma de pipeline explícito. Cada iteración llama a `sha256_compress` dos veces secuencialmente, por lo que cada nonce tarda aproximadamente $2 \times 64 = 128$ ciclos de reloj.

EN: *The top-level mining loop has no explicit pipeline pragma. Each iteration calls sha256_compress twice sequentially, so each nonce takes approximately* $2 \times 64 = 128$ *clock cycles.*

6.3 Configuraciones de Solución / Solution Configurations

Table 4: Configuraciones de solución HLS / HLS Solution Configurations

	Solución / Solution	Compress II	Ciclos/nonce	Throughput
2palebluewhite	1: Baseline (II=1)	1	∼128	∼780 KH/s
	2: Relaxed (II=2)	2	∼256	∼390 KH/s

ES: La Solución 2 utiliza una directiva Tcl para relajar el II=1 del código fuente:

EN: *Solution 2 uses a Tcl directive to override the source-level II=1:*

```
set_directive_pipeline -II 2 "sha256_compress/compress"
```

ES: Esto proporciona más margen de timing si la Solución 1 no cumple la restricción de 10 ns a 100 MHz.

EN: *This provides more timing margin if Solution 1 fails to meet the 10 ns constraint at 100 MHz.*

7 Bloque 939260 – Demostración / Block 939260 – Demo

ES: El bloque seleccionado para la demostración es el Bloque 939260 de Bitcoin, minado el 4 de marzo de 2026. Sus características lo hacen apropiado para una demostración en clase:

EN: *The block selected for the demo is Bitcoin Block 939260, mined on March 4, 2026. Its characteristics make it suitable for an in-class demonstration:*

Table 5: Datos del Bloque 939260 / Block 939260 Data

	Campo / Field	**Valor / Value**
2palebluewhite	Altura / Height	939,260
	Fecha / Date	4 marzo / March 4, 2026
	Dificultad / Difficulty	144,398,401,518,101
	Nonce (LE)	`0x1E740A08`
	Nonce (BE)	`0x080A741E` (∼134.9M)
	Block hash	`0000000000000000000001758847...`
	Tiempo est. / Est. time	∼3 min at 781 KH/s

ES: **Cabecera cruda** (hex, 80 bytes):

`0000003c 3c08f14a ad5cac28 2d1c8ca8 79310a24 15794de3 8e4e0100 00000000 00000000 fc9099f6 927218e1 2db0cc20 ef43657d 5a32cba9 3b8994ba 59a61054 ddf1af1b bf28a869 03f30117 080a741e`

Hash del bloque (orden de visualización):

`0000000000000000000017588478b3612182486d33006ede1164fb146fa41cd06`

EN: *The raw header hex and block hash are shown above. The nonce BE value 0x080A741E corresponds to approximately 134.9 million iterations from nonce=0, resulting in a mining time of approximately 3 minutes at the estimated 781 KH/s throughput.*

7.1 Orden de Bytes del Nonce / Nonce Byte Order

ES: Bitcoin serializa los nonces en **little-endian** (LE) en la cabecera del bloque. SHA-256 procesa palabras en **big-endian** (BE). La FPGA itera directamente la palabra de nonce en BE. La FPGA busca en un orden diferente a un minero LE, pero cubre el mismo espacio de 2^{32} nonces y produce hashes idénticos para cualquier valor de nonce dado.

EN: *Bitcoin serializes nonces in little-endian (LE) in the block header. SHA-256 processes words in big-endian (BE). The FPGA iterates the BE nonce word directly, searching in a different order than a LE miner but covering the same 2^{32} nonce space and producing identical hashes for any given nonce value.*

8 Verificación y Pruebas / Verification and Testing

8.1 Testbench HLS (5 Pruebas) / HLS Testbench (5 Tests)

ES: El testbench (`pico_miner_tb.cpp`) contiene cinco casos de prueba:

EN: *The testbench (pico_miner_tb.cpp) contains five test cases:*

Table 6: Casos de prueba del testbench / Testbench test cases

2palebluewhite

Test	Tipo / Type	Descripción / Description
1	NIST vector	SHA-256("abc") validado contra FIPS 180-4
2	Block 1 SW	Minería completa desde nonce=0 (∼31.8M iteraciones) en software
3	Block 1 HW	Acelerador HLS con ventana ±16 nonces
4	Block 939260 HW	Acelerador HLS con ventana ±16 nonces
5	Sin solución	Verifica reporte "no encontrado" / "not found"

ES: Las ventanas estrechas de ±16 nonces para las pruebas HW mantienen la co-simulación C/RTL rápida (32 nonces × 128 ciclos/nonce ≈ 4096 ciclos por bloque). La búsqueda completa en el Test 2 se ejecuta en software puro para evitar timeouts en la co-simulación.

EN: *The narrow ±16 nonce windows for HW tests keep C/RTL co-simulation fast (32 nonces × 128 cycles/nonce ≈ 4096 cycles per block). The full-range search in Test 2 runs in pure software to avoid cosim timeout.*

8.2 Simulación C (csim) / C Simulation (csim)

ES: Las cinco pruebas se ejecutan durante `csim_design`. La búsqueda completa del Bloque 1 (Test 2) tarda ∼10 segundos en simulación C.

EN: *All five tests run during csim_design. The full-range Block 1 search (Test 2) takes approximately 10 seconds in C simulation.*

8.3 Co-Simulación C/RTL (cosim) / C/RTL Co-Simulation (cosim)

ES: Tras la síntesis C, el RTL generado se verifica contra el testbench C. Las llamadas HW usan ventanas estrechas de nonces (±16) para mantener el tiempo de co-simulación manejable.

EN: *After C synthesis, the generated RTL is verified against the C testbench. HW calls use narrow nonce windows (±16) to keep cosim runtime manageable.*

8.4 Verificación en ARM / ARM Verification

ES: El driver ARM (`pico_miner_arm.c`) mina el Bloque 939260 en hardware FPGA mediante búsqueda completa desde nonce=0. Tras encontrar un nonce, el ARM:

1. Ejecuta el modelo dorado SW (double-SHA-256 completo) sobre el nonce encontrado
2. Muestra el hash calculado en orden de visualización de Bitcoin
3. Compara el nonce encontrado con el nonce conocido de la blockchain
4. Informa del estado de verificación por terminal UART

EN: *The ARM driver mines Block 939260 on FPGA hardware via full brute-force search from nonce=0. After finding a nonce, the ARM runs the SW golden model, displays the computed hash in Bitcoin display order, compares against the known blockchain nonce, and reports verification status on the UART terminal.*

9 Resultados / Results

9.1 Rendimiento Estimado / Estimated Performance

ES: A 100 MHz con compresión II=1, la latencia de compresión SHA-256 es de 64 ciclos. Cada nonce requiere dos compresiones, resultando en ∼128 ciclos por nonce:

$$\text{Throughput} = \frac{100 \times 10^6 \text{ ciclos/s}}{128 \text{ ciclos/nonce}} \approx 781{,}250 \text{ H/s} \approx 781 \text{ KH/s}$$

EN: *At 100 MHz with II=1 compress, SHA-256 compression latency is 64 cycles. Each nonce requires two compressions, resulting in approximately 128 cycles per nonce, yielding approximately 781 KH/s throughput.*

9.2 Tiempo de Minería del Bloque 939260 / Block 939260 Mining Time

ES: El nonce BE del Bloque 939260 es `0x080A741E` (≈134.9M). A ∼781 KH/s:

$$\text{Tiempo} = \frac{134{,}900{,}000}{781{,}000} \approx 172.7 \text{ s} \approx 2 \text{ min } 53 \text{ s}$$

Esto constituye un tiempo de demostración apropiado para el aula.

EN: *Block 939260's BE nonce is 0x080A741E (approximately 134.9M). At approximately 781 KH/s, the estimated mining time is approximately 173 seconds (about 2 minutes 53 seconds), which is a suitable demo length for the classroom.*

9.3 Comprobación de Dificultad / Difficulty Check

ES: El hash del Bloque 939260 comienza con 19 ceros hexadecimales (76 bits a cero) en orden de visualización. La comprobación simplificada `final_hash[7] == 0` verifica que los primeros 32 bits en orden de visualización son cero. Aunque esto es menos restrictivo que la dificultad real (que requiere 76+ bits a cero), es suficiente para la demostración. La probabilidad de un falso positivo (`final_hash[7] == 0` para un nonce aleatorio) es $1/2^{32} \approx 2.3 \times 10^{-10}$, acumulando ∼3% sobre los 134.9M nonces buscados.

EN: *Block 939260's hash starts with 19 hex zeros (76 zero bits) in display order. The simplified check final_hash[7] == 0 verifies that the first 32 display-order bits are zero. While less restrictive than the actual difficulty, it is sufficient for the demo. The false positive probability is approximately 3% over the 134.9M nonces searched.*

10 Guía de Reproducción / Reproduction Guide

10.1 Paso 1: Vivado HLS / Step 1: Vivado HLS

ES: Sintetizar el IP de minería:

EN: *Synthesize the mining IP:*

```
vivado_hls -f run_hls.tcl
```

ES: Alternativamente, en la GUI de Vivado HLS:

1. Crear nuevo proyecto, función top `pico_miner`
2. Añadir `src/pico_miner.cpp` y `src/pico_miner.h` como archivos fuente
3. Añadir `src/pico_miner_tb.cpp` como testbench
4. Establecer dispositivo objetivo: `xc7z020clg484-1`, reloj: 10 ns (100 MHz)
5. Ejecutar C Simulation → debe mostrar "ALL TESTS PASSED (5 tests)"
6. Ejecutar C Synthesis → genera RTL + informe de rendimiento
7. Ejecutar C/RTL Co-Simulation → verifica que el RTL coincide con el comportamiento C
8. Exportar RTL (formato IP Catalog)

EN: *Alternatively, in the Vivado HLS GUI: create a new project with top function pico_miner, add source and testbench files, set target part xc7z020clg484-1 with 10 ns clock, then run C Simulation, C Synthesis, C/RTL Co-Simulation, and Export RTL.*

10.2 Paso 2: Vivado – Diseño de Bloques / Step 2: Vivado – Block Design

ES:

1. Abrir Vivado 2019.1, crear proyecto para `xc7z020clg484-1`
2. Crear un Block Design
3. Añadir el IP **ZYNQ7 Processing System**
4. Añadir el IP **Pico Miner** (desde la exportación HLS)
5. Añadir un IP **AXI GPIO**, configurar como salida de 8 bits (LEDs)
6. Mantener `FCLK_CLK0 = 100 MHz` (por defecto)
7. Ejecutar **Connection Automation**
8. Conectar la salida GPIO a los pines de LED del ZedBoard (restricciones XDC)
9. Generar bitstream
10. Exportar Hardware (incluir bitstream) y lanzar SDK

EN: *Open Vivado, create project for xc7z020clg484-1, create Block Design, add Zynq PS, Pico Miner IP, AXI GPIO for LEDs, run Connection Automation, generate bitstream, export hardware with bitstream, and launch SDK.*

10.3 Paso 3: Xilinx SDK / Step 3: Xilinx SDK

ES:

1. Crear un nuevo Application Project (plantilla **Empty Application**)
2. Añadir `src/pico_miner_arm.c` a la carpeta `src/`
3. Compilar el proyecto
4. Programar la FPGA y ejecutar la aplicación
5. Observar resultados en el terminal serie UART (115200 baudios)
6. Observar los LEDs del ZedBoard encendiéndose progresivamente

Nota: Usar la plantilla "Empty Application", no "Hello World". El driver usa `xil_cache.h` directamente (no `platform.h`).

EN: *Create a new Application Project (Empty Application template), add pico_miner_arm.c, build, program FPGA and run. Observe UART output (115200 baud) and LED progress on the ZedBoard. Use "Empty Application" template — the driver uses xil_cache.h directly, not platform.h.*

11 Conclusión / Conclusion

ES: Pico Miner demuestra la minería Bitcoin double-SHA-256 con optimización de midstate en una FPGA Zynq-7020 mediante Vivado HLS 2019.1. El sistema realiza una búsqueda por fuerza bruta del Bloque 939260 (4 de marzo de 2026, dificultad 144.398.401.518.101) desde nonce=0, encontrando el nonce correcto en aproximadamente 3 minutos.

El diseño se verifica a múltiples niveles:

- Validación contra vector de prueba NIST FIPS 180-4
- Minería completa del Bloque 1 desde nonce=0 (∼31.8M iteraciones) en software
- Minería hardware de los Bloques 1 y 939260 con comparación bit-exacta SW/HW
- Hashes de bloque verificados contra la blockchain
- Verificación SW del nonce encontrado por hardware en el driver ARM

El proyecto demuestra:

- Síntesis de Alto Nivel (HLS) de SHA-256 desde C hasta RTL
- Optimización de midstate para partición de trabajo ARM/FPGA
- Integración AXI-Lite para comunicación PS/PL
- Optimización mediante directivas HLS (pipeline II=1, partición de arrays, desenrollamiento)
- Control de GPIO para retroalimentación visual (LEDs)
- Medición de rendimiento en tiempo real (XTime)

El throughput estimado es de ∼781 KH/s a 100 MHz con compresión II=1 (∼128 ciclos por nonce).

EN:

The design is verified at multiple levels: NIST FIPS 180-4 test vector validation, full-range Block 1 mining from nonce=0, hardware mining of Blocks 1 and 939260 with bit-exact SW/HW comparison, and SW verification of the hardware-found nonce in the ARM driver.

The project showcases High-Level Synthesis of SHA-256 from C to RTL, midstate optimization for ARM/FPGA work partitioning, AXI-Lite PS/PL integration, HLS directive optimization (pipeline II=1, array partition, unroll), GPIO LED control, and real-time performance measurement via XTime.

Estimated throughput is approximately 781 KH/s at 100 MHz with II=1 compress (approximately 128 cycles per nonce).*Pico Miner demonstrates Bitcoin double-SHA-256 mining with midstate optimization on a Zynq-7020 FPGA using Vivado HLS 2019.1. The system performs a full brute-force search of Block 939260 (March 4, 2026, difficulty 144,398,401,518,101) from nonce=0, finding the correct nonce in approximately 3 minutes.*

The design is verified at multiple levels: NIST FIPS 180-4 test vector validation, full-range Block 1 mining from nonce=0, hardware mining of Blocks 1 and 939260 with bit-exact SW/HW comparison, and SW verification of the hardware-found nonce in the ARM driver.

The project showcases High-Level Synthesis of SHA-256 from C to RTL, midstate optimization for ARM/FPGA work partitioning, AXI-Lite PS/PL integration, HLS directive optimization (pipeline II=1, array partition, unroll), GPIO LED control, and real-time performance measurement via XTime.

Estimated throughput is approximately 781 KH/s at 100 MHz with II=1 compress (approximately 128 cycles per nonce).

12 Archivos del Proyecto / Project Files

Table 7: Archivos del proyecto / Project files

2palebluewhite

Archivo / File	**Descripción / Description**
`src/pico_miner.h`	Constantes SHA-256 (H0-H7, K[64]), prototipo de función, defines de interfaz / SHA-256 constants, function prototype, interface defines
`src/pico_miner.cpp`	Fuente HLS: `sha256_compress()` + bucle de minería double-SHA-256 con midstate / HLS source: compress + mining loop
`src/pico_miner_tb.cpp`	Testbench HLS: 5 pruebas (NIST, Bloque 1 SW+HW, Bloque 939260 HW, sin solución) / 5 tests
`src/pico_miner_arm.c`	Driver ARM: demo Bloque 939260 con progreso, LEDs, XTime / ARM driver: full-range demo
`doc/poc.tex`	Este documento: memoria del proyecto bilingue ES/EN / This document
`run_hls.tcl`	Script TCL: 2 soluciones (II=1, II=2), csim/csynth/cosim/export